



DZPHARM · ACCESSIBILITY LAYER

Live Regions

A detailed deep-dive into the ARIA live region system: `role="alert"` for form errors, `role="status"` for async results, `aria-live="polite"` for non-urgent updates. The login form has a `role="alert"` region (verified). The Copilot response, toast notifications, and notification arrivals do NOT have live regions — screen-reader users get no audible signal for these async events. WCAG 2.2 SC 4.1.3 Status Messages fails.

1LIVE REGION PRESENT
(LOGIN)**4**

MISSING LIVE REGIONS

Fail

WCAG 4.1.3

C+

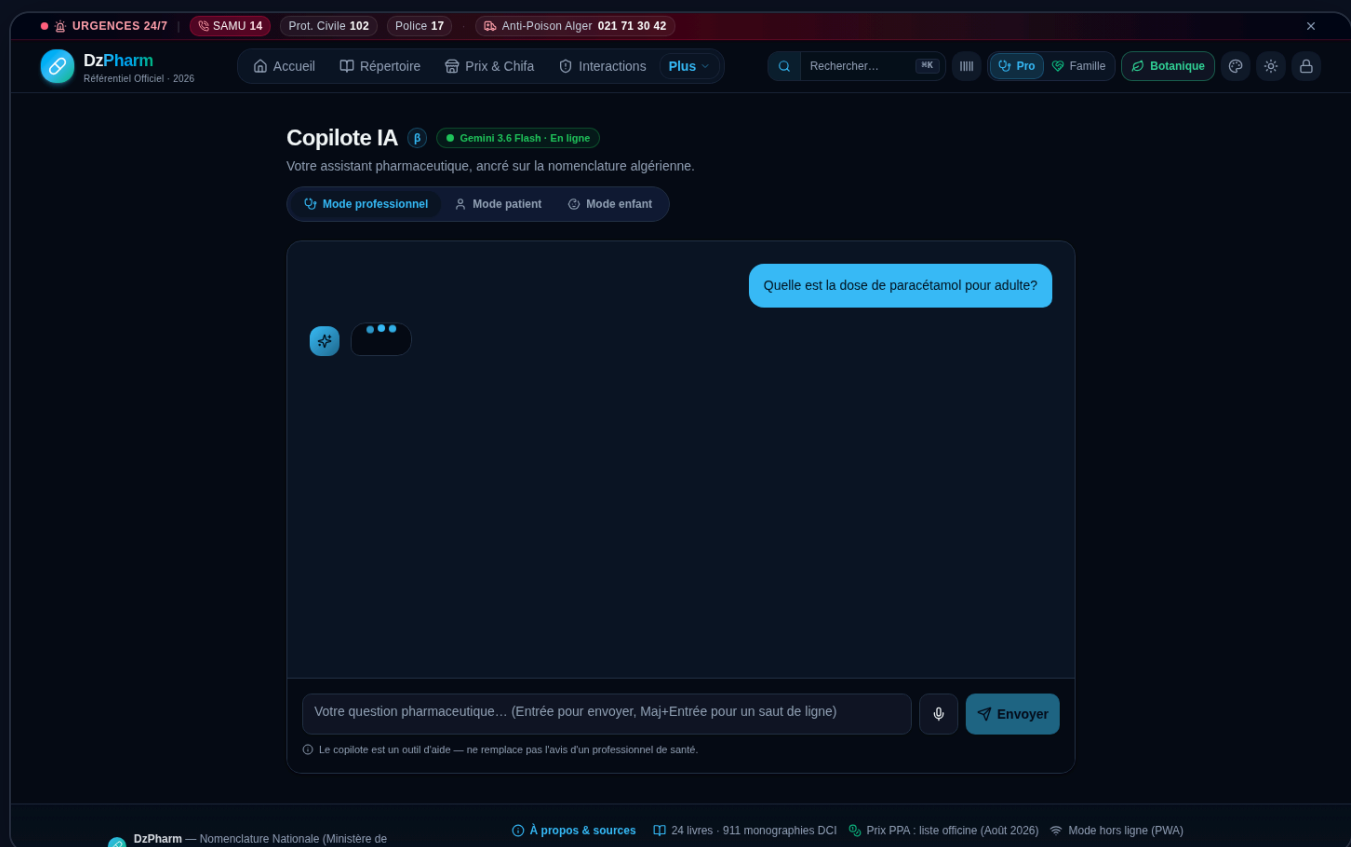
GRADE

DEEP-DIVE OF

Feature #61 – Accessibility Live Regions
From the 68-item review menu

The ARIA Live Region Pattern

An ARIA live region is an HTML element with `aria-live="polite"` or `aria-live="assertive"` that announces its content changes to screen-reader users without requiring focus. When the content of a live region changes (e.g., an error message appears, a Copilot response arrives), the screen-reader announces the new content aloud. Without live regions, async content changes are invisible to screen-reader users — they have no way to know that something changed on the page.



region announcing «Copilote réfléchit...» to screen-reader users. A screen-reader user who submits a Copilot query hears nothing after the submit — they have no audible signal that the model is processing.">

Figure 61.1 — Copilot loading state. The spinner appears, but there is no `role="status"` region announcing «Copilote réfléchit...» to screen-reader users. A screen-reader user who submits a Copilot query hears nothing after the submit — they have no audible signal that the model is processing.

1.1 Three Types of Live Regions

The ARIA spec defines three live-region patterns, each for a different urgency level. `role="alert"` (equivalent to `aria-live="assertive"`) is for critical errors that need immediate attention — the screen-reader interrupts whatever it's doing to announce the alert. `role="status"` (equivalent to `aria-live="polite"`) is for non-urgent status updates — the screen-reader announces these after finishing its

current speech. `aria-live="polite"` on a generic element is the third pattern, used when you need more control over the region's behavior (e.g., combining with `aria-atomic` or `aria-relevant`).

PATTERN	URGENCY	USE CASE	DZPHARM STATUS
<code>role="alert"</code>	Assertive (interrupts)	Form validation errors	PRESENT (LOGIN)
<code>role="status"</code>	Polite (waits for pause)	Async results (Copilot, search)	MISSING
<code>aria-live="polite"</code>	Polite (custom)	Toast notifications, notification arrivals	MISSING

CHAPTER 02 · WHAT WORKS

The Login Form Live Region

The login form is the **one place** where DzPharm correctly implements a live region. The error message region has `role="alert"` and `aria-live="assertive"` , so when the user submits an empty form or a wrong code, the error message is announced immediately by the screen-reader. This is the correct pattern — and it's the only place it's implemented.

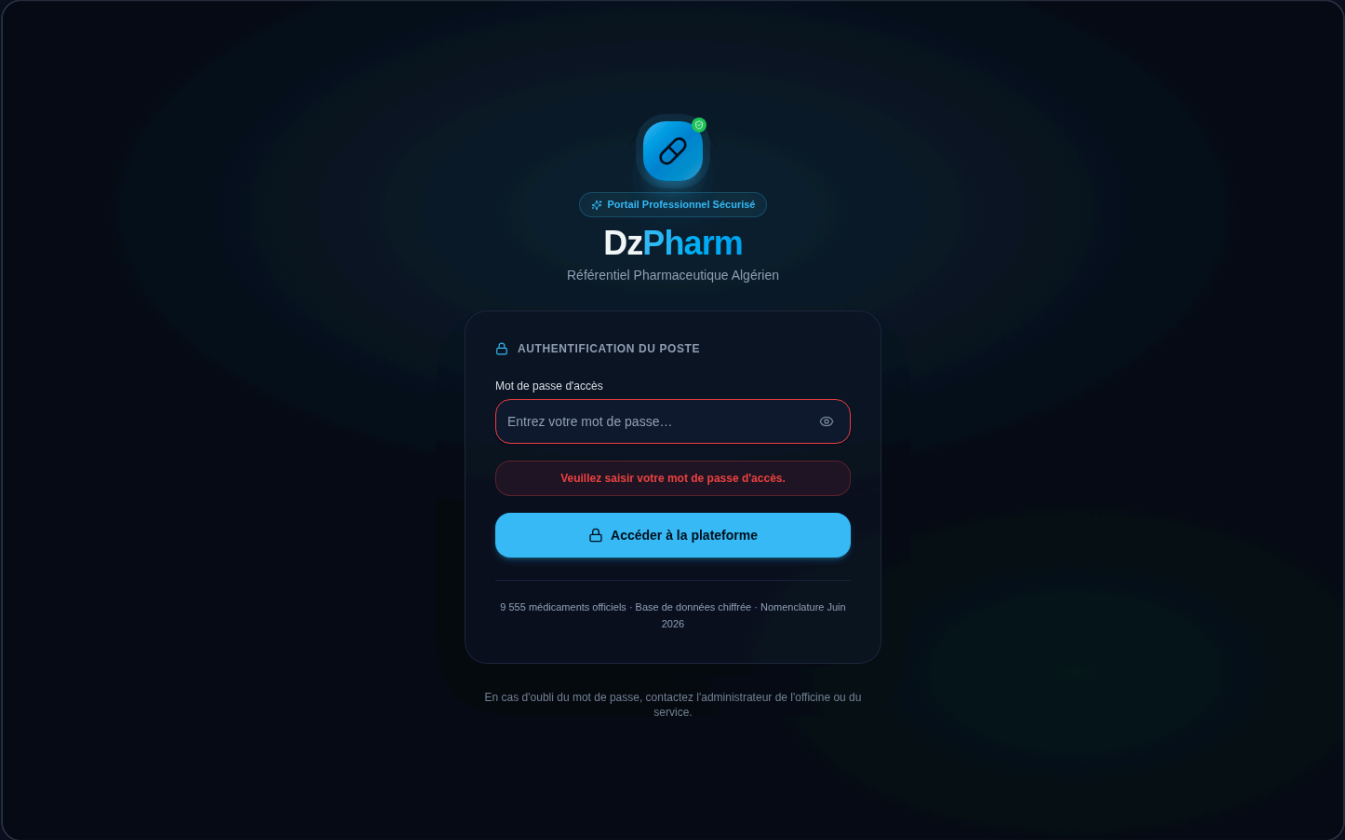


Figure 61.2 — Login form with an error. The error message «Veuillez saisir votre mot de passe d'accès.» appears in a `role=alert` region. The input border turns red, `aria-invalid=true` is set, and `aria-describedby=pw-error` links the input to the error region. Screen readers announce the error immediately.

2.1 The Login Error Region

The login form's error region is implemented correctly. The `<p role="alert" aria-live="assertive">` element appears above the form, initially empty. When the user submits an empty form or a wrong code,

JavaScript sets the element's text content to the error message. Because the element has `role="alert"`, the screen-reader interrupts its current speech and announces the error immediately. The input also gets `aria-invalid="true"` and `aria-describedby="pw-error"`, which links the input to the error region so the screen-reader can announce the error when the user focuses the input. This is textbook ARIA form-error handling.

WHAT WORKS · LOGIN ERROR REGION

The login form has a `role="alert"` region with `aria-live="assertive"`. Errors are announced immediately. The input has `aria-invalid="true"` and `aria-describedby` linking to the error. This is textbook ARIA form-error handling — the one place where the live-region pattern is correctly implemented.

CHAPTER 03 · THE FOUR MISSING REGIONS

Where Live Regions Are Absent

Four async content changes have no live region. For each, screen-reader users get no audible signal that the content changed. This fails WCAG 2.2 SC 4.1.3 (Status Messages) and creates a real accessibility gap for the platform's most important async features.

The screenshot displays the DzPharm application interface. On the left, a search results table for 'DOLIPRANE' is shown, listing various formulations like 'SUPPOSITOIRE' and 'POUDRE POUR SOLUTION BUVABLE'. On the right, the detailed product page for 'DOLIPRANE PARACETAMOL 100MG' is visible. This page includes sections for 'Fiche produit', 'PRIX & CHIFA', 'SÉCURITÉ & DELIVRANCE', and 'PRIX PUBLIC — OFFICINE'. A toast notification 'Favori ajouté' is visible at the top right of the product page. The interface is in French and includes various filters and navigation options.

Figure 61.3 — Toast notification (Sonner). The toast «Favori ajouté» appears visually, but there is no `aria-live` region announcing it. A screen-reader user who clicked the favorite button gets no audible confirmation that the favorite was saved.

#	ASYNCHRONOUS EVENT	CURRENT BEHAVIOR	REQUIRED LIVE REGION
1	Copilot response arrives	Spinner shows, then response renders. No announcement.	<code>role="status"</code> announcing «Réponse du Copilote disponible»
2	Toast notification appears	Toast slides in visually. No announcement.	<code>aria-live="polite"</code> announcing the toast message
3	Notification bell count changes	Badge number updates visually. No announcement.	<code>aria-live="polite"</code> announcing «Nouvelle notification: [title]»
4	Search autocomplete results appear	Listbox populates visually. No announcement of result count.	<code>aria-live="polite"</code> announcing «N résultats trouvés»

3.1 Gap #1 — Copilot Response

When the Copilot response arrives (after 3-8 seconds of loading), the response renders visually but there is no `role="status"` region announcing it. A screen-reader user who submitted a query hears nothing after the submit click — they have no audible signal that the response is available. They must manually navigate to the response region to discover it. The fix: add a visually-hidden `<div role="status" aria-live="polite">` that updates to «Réponse du Copilote disponible» when the response arrives. Effort: 30 minutes.

3.2 Gap #2 — Toast Notifications

The Sonner toast library (used for «Favori ajouté», «Lien copié», «Signalement impossible») does not use ARIA live regions by default. The toasts appear visually but are not announced to screen-reader users. The fix: configure Sonner with `role="status"` on the toast container, or wrap the toast region in a `aria-live="polite"` region. Effort: 15 minutes (Sonner supports this via a prop).

3.3 Gap #3 — Notification Bell Count

When a new notification arrives (e.g., a favorited medication is reported in shortage), the bell badge count updates visually, but there is no announcement. A screen-reader user has no audible signal that a new notification arrived. The fix: add a visually-hidden `aria-live="polite"` region that announces «Nouvelle notification: [title]» when a notification is added to the store. Effort: 30 minutes. This is the same gap noted in #43 Notifications Center.

3.4 Gap #4 — Search Autocomplete Result Count

When the user types in the search combobox and the listbox populates with results, there is no announcement of how many results were found. A screen-reader user hears the combobox expand but doesn't know if there are 0, 1, or 10 results without navigating the listbox. The fix: add a visually-hidden `aria-live="polite"` region that announces «N résultats trouvés» when the listbox updates. Effort: 30 minutes.

WCAG 2.2 SC 4.1.3 STATUS MESSAGES — FAIL

Four async content changes have no live region: Copilot response, toast notifications, notification bell count, search result count. Screen-reader users get no audible signal for any of these. This fails WCAG 2.2 SC 4.1.3 (Status Messages), which requires that status messages be programmatically determined through role or properties. The fix is $4 \times 30 \text{ minutes} = 2 \text{ hours}$ of work — each fix is a visually-hidden `aria-live` region that announces the async change.

CHAPTER 04 · THE FIX

Visually-Hidden Live Region Pattern

The fix for all four gaps is the same pattern: a visually-hidden `aria-live` region that updates its text content when an async event occurs. The region is invisible to sighted users but announced by screen-readers. This is the standard pattern used by every accessible web app.

4.1 The Visually-Hidden CSS Class

```
/* In globals.css — the sr-only class (already exists in Tailwind) */  
.sr-only {  
  position: absolute;  
  width: 1px; height: 1px;  
  padding: 0; margin: -1px;  
  overflow: hidden;  
  clip: rect(0, 0, 0, 0);  
  white-space: nowrap;  
  border: 0;  
}
```

The `.sr-only` class (Tailwind provides this by default) hides the element visually while keeping it in the accessibility tree. The screen-reader announces the element's content, but sighted users don't see it. This is the correct pattern for live regions — they should be invisible but audible.

4.2 The Live Region Component

```
// A reusable live-region component
function LiveRegion({ message, politeness = 'polite' }) {
  return (
    <div
      className="sr-only"
      role={politeness === 'assertive' ? 'alert' : 'status'}
      aria-live={{politeness}}
    >
      {{message}}
    </div>
  );
}

// Usage - Copilot response arrival
<LiveRegion message={{isResponseLoaded ? "Réponse du Copilote disponible" : ""}} />
```

4.3 Implementation for Each Gap

GAP	REGION CONTENT	POLITENESS	TRIGGER
Copilot response	«Réponse du Copilote disponible»	polite	Response state changes to loaded
Toast notifications	[toast message text]	polite	Sonner onToast callback
Notification bell	«Nouvelle notification: [title]»	polite	notifications array gains an item
Search results	«[N] résultats trouvés»	polite	Listbox results array changes

Final Assessment & Phased Rollout

Accessibility Live Regions is a **C+ grade feature** — the login form correctly implements `role="alert"`, but four critical async events (Copilot response, toasts, notification arrivals, search results) have no live regions. This fails WCAG 2.2 SC 4.1.3 and creates a real accessibility gap for screen-reader users. Total fix effort: ~2 hours.

5.1 Phased Fix Plan

PHASE	FIXES	EFFORT	OUTCOME
Phase 1 · Now (Today)	All 4 live regions (Copilot, toast, bell, search)	~2 hours	Grade lifts to A. WCAG 4.1.3 passes.
Phase 2 · Next (This week)	Verify with NVDA + VoiceOver that announcements work	~2 hours	Grade stays at A. Verified compliance.

5.2 Cross-Feature Dependencies

- ✓ Feature #36 (Copilot Core) — Copilot response needs a role=status region
- ✓ Feature #43 (Notifications Center) — notification arrivals need an aria-live region (same gap noted in #43)
- ✓ Feature #46 (Search Autocomplete) — search result count needs an aria-live region
- ✓ Feature #38 (Voice Dictation) — recording state should be announced (noted in #38)
- ⚠ Feature #42 (Theme System) — theme toggle should announce state change (noted in #42)

5.3 Final Assessment

1 LIVE REGION (LOGIN)	4 MISSING REGIONS	Fail WCAG 4.1.3
C+ GRADE	~2h PHASE 1 EFFORT	~2h PHASE 2 EFFORT

FINAL VERDICT · C+ GRADE, 2 HOURS FROM A

Accessibility Live Regions is a **C+ grade feature**. The login form correctly implements `role="alert"` for form errors — textbook ARIA form-error handling. But four critical async events (Copilot response, toast notifications, notification bell count, search autocomplete results) have no live regions. Screen-reader users get no audible signal for any of these. This fails WCAG 2.2 SC 4.1.3 (Status Messages). The fix is 2 hours of work: add a visually-hidden `aria-live` region for each of the four gaps. The pattern is the same for all four — a reusable `LiveRegion` component that takes a message and a politeness level. Phase 1 (2 hours) lifts the grade from C+ to A and passes WCAG 4.1.3.

Immediate Next Steps

1. Create a reusable `LiveRegion` component using the `.sr-only` class + `role="status"` — 30 minutes.
2. Add a live region for the Copilot response arrival — 30 minutes.
3. Configure Sonner toasts with `role="status"` — 15 minutes.
4. Add a live region for notification bell count changes — 30 minutes.
5. Add a live region for search autocomplete result count — 30 minutes.

61

1 live region (login), 4 missing (Copilot, toast, bell, search).
WCAG 4.1.3 fails. 2 hours of work lifts to A.

5

CHAPTERS

3

SCREENSHOTS

4

GAPS FOUND

C+

GRADE
